

Vector Core for Accelerating Nonlinear Layers of AI Models in the Atalla AI Chip

Julio Hernandez VIP 47922 herna628@purdue.edu	Jacob Walter VIP 47922 walte250@purdue.edu
Vedant Sharma VIP 37900 sharm792@purdue.edu	Mayank Patibandla VIP 47921 mpatiban@purdue.edu
Akhil Yada VIP 47921 yadaa@purdue.edu	Brian Zhuang VIP 37900 bzhuang@purdue.edu

Spring 2026

The feedback for the report is to be returned to:

Akhil Yada
yadaa@purdue.edu

This project was mentored by the following Graduate Teaching Assistants:

Fan Jing Hoon, hoonf@purdue.edu
Sooraj Chetput, schetput@purdue.edu

Vector Core for Accelerating Nonlinear Layers of AI Models in the Atalla AI Chip

Julio Hernandez
VIP 47922
hern628@purdue.edu

Mayank Patibandla
VIP 47921
mpatiban@purdue.edu

Jacob Walter
VIP 47922
walte250@purdue.edu

Akhil Yada
VIP 47921
yadaa@purdue.edu

Vedant Sharma
VIP 37900
sharm792@purdue.edu

Brian Zhuang
VIP 37900
bzhuang@purdue.edu

Abstract

AI models rely on matrix multiplication for dense compute and on element-wise nonlinear activation functions for representing complex patterns within the model. Element-wise activations expose data-level parallelism (DLP) that a single-instruction, multiple-data (SIMD) datapath can convert into linear throughput scaling by having parallel lanes of high-throughput arithmetic units that process independent vector elements simultaneously. The prior Atalla Vector Core instantiated an unpipelined iterative divider in each of its 16 lanes and a multi-cycle FSM-sequenced exponential unit, both of which issued at rates of 11 and 14 cycles respectively, throttling vector throughput. Matrix transpose was performed by swizzling inside the scratchpad, whose crossbar incurred large area and whose banking constraints prevented wider per-bank read ports. To address these bottlenecks, we replace the per-lane dividers with a single 9-stage pipelined reciprocal unit in the scalar core, redesign the exponential unit as a single-cycle LUT-based datapath, and introduce a dedicated transpose unit. Synthesized on FreePDK 45-nm at 800 MHz, the reciprocal unit reaches a 0.65 average ULP (1 ULP worst case) in $5,064 \mu m^2$, and the exponential unit reaches a 0.03 average ULP (2 ULP worst case) in $1,798 \mu m^2$, both at an effective CPI of 1.0. The transpose unit requires 32 sequential pops per matrix, creating a pipeline bubble between consecutive transpositions on partially-filled tiles. Future work uses per-unit utilization and stall data from a testbench performance monitor to guide further optimization, and extends the datapath with MxN and pooling support for broader AI workload coverage.

1 Introduction

To meet the computational demands of AI models, the Atalla Ax01 processor is structured around four major units. The Scalar Core handles control flow and scalar arithmetic. The VLIW Scheduler decodes and issues instructions to the respective units. The Systolic Array performs GEMM. The Vector Core executes element-wise operations in parallel across vector data using a 16-lane SIMD datapath to exploit DLP. Data is stored on-chip in Scratchpad memories and in a Vector Register File (VRF) which provides high-bandwidth access to intermediate operands for the Vector Core. This architecture allows the nonlinear operations that sit between GEMM passes to be executed without moving data off-chip.

1.1 Background

The previous iteration of the Atalla Vector Core established the SIMD datapath which utilized a 16-lane architecture capable of processing 32-element vectors. It integrated a Vector Register File (VRF) to hold vector operands and Systolic Array tiles, Vector Load Store Units (VLSU) to decouple memory requests to the Scratchpad, and a Global Systolic Array Unit (GSAU) to interface with the Systolic Array. Rather than employing a single set of wide functional units, the architecture distributed execution across 16 parallel lanes. This lane-based approach was chosen to maximize scalability and eliminate the need for an area-intensive crossbar network. While this baseline demonstrated DLP, its performance and area efficiency were limited by the specific implementations of its functional units and memory subsystems. The arithmetic functional units, specifically the divider and exponential units, lacked pipelining (issue rates of 11 and 14 cycles respectively). Back-to-back divisions or exponentiations serialized, throttling vector throughput on workloads dominated by these operations, such as Softmax and Sigmoid. Additionally, data movement operations, such as matrix transpositions, were coupled directly to the scratchpad memory. This coupling forced the scratchpad to maintain $NUM_BANKS = MVL$ so that column-major reads could be produced via bank-level swizzling, preventing the use of wider per-bank ports for higher memory bandwidth.

1.2 Common Concepts

The following concepts are referenced throughout this report and are briefly summarized for readers unfamiliar with the architecture.

SIMD/Lane Architecture. The Vector Core uses a SIMD execution model in which a single instruction operates on all 32 elements of a vector register simultaneously. The work is distributed across 16 parallel lanes, each processing a 2-element slice of the vector. This eliminates the need for a global crossbar over full vector operands, as each lane only sees the data it needs. For element-wise operations, throughput scales linearly with the number of lanes.

BF16 and ULP. All arithmetic within the Vector Core uses the BF16 (Brain Floating Point 16) format [1], a 16-bit floating-point representation with 1 sign bit, 8 exponent bits, and 7 mantissa bits. BF16 provides the same dynamic range as IEEE FP32 while halving storage and bandwidth requirements. Precision is measured in ULP (Unit in the Last Place), the gap between two adjacent representable floating-point val-

ues. A result with 0 ULP is exact and 1 ULP is off by one representable value.

Valid/Ready Handshake. The Vector Core uses a valid/ready handshake protocol, based on the ARM AMBA AXI specification [2], to synchronize data transfers between units with varying latencies. An upstream producer asserts `valid` when its output is ready to be consumed; a downstream consumer asserts `ready` when it can accept new data. A transfer occurs only when both signals are high. This protocol provides backpressure, preventing data loss when a downstream unit is busy.

1.3 Target Workloads

Different AI model architectures rely on specific activation profiles that dictate the utilization of the Vector Core’s functional units. Table 1 summarizes representative workloads, their structural characteristics, and the vector operations they demand.

These operations are highly data-parallel and directly dictate the architecture of the Vector Core. For example, Transformer-based architectures [4] heavily utilize the exponential and reciprocal units for Softmax, require square root and reciprocal operations for LayerNorm, and demand matrix transpositions before computing attention scores (QK^T), which directly motivates the dedicated Transpose Unit.

1.4 Contributions

Since the foundational components and their design decisions were established last semester, this report focuses exclusively on the architectural modifications and new additions designed to resolve the aforementioned bottlenecks. Specifically, we present the design and evaluation of a reciprocal unit that replaces the per-lane dividers with a single pipelined scalar-core instance, the relocation of the per-lane square root unit to the scalar core, a redesigned exponential unit, and a dedicated SRAM-based transpose unit that decouples transposition from the scratchpad memory. We also complete the implementation of the Vector Load Store Unit (VLSU) from prior-semester design work.

2 Design Choices

2.1 Top Level Architecture

This semester, the Vector Core underwent several architectural changes from the prior design to improve performance, reduce area, and support system integration. The updated top-level architecture is shown in fig. 1, and the key changes are:

- The per-lane divider and square root units were removed from the vector lanes and replaced with a single reciprocal unit and square root unit in the Scalar Core. Since these operations compute a single scalar result broadcast to the lanes for multiplication, duplicating them across

16 lanes was unnecessary. This change reduces lane area while maintaining throughput for the common case.

- To simplify the writeback interface, result collection was moved out of each lane into per-functional-unit global collectors.
- To streamline data fetching and writeback, the Vector Register File (VRF) control was moved to the Scheduler. This allowed for smoother development and integration with the Scheduler.
- The Scratchpad count was increased from 2 to 4, providing additional independent memory channels for the Vector Core.
- The exponential unit within each lane was redesigned with a LUT-based single-cycle architecture to replace the prior multi-cycle iterative design.
- A dedicated Transpose Unit was added, decoupled from the Scratchpad, to support matrix transposition without consuming memory bandwidth.

The following subsections detail the rationale and trade-offs for each major design choice.

2.2 Reciprocal Unit

The reciprocal unit was created to support operations that require division by a scalar denominator (e.g., Softmax, LayerNorm). Originally, a full divider was instantiated in each of the 16 vector lanes. However, division in these workloads always takes the form of dividing an entire vector by a single scalar. This makes per-lane dividers unnecessary. The scalar denominator’s reciprocal can be computed once in the Scalar Core and broadcast to the lanes, where the existing per-lane multipliers complete the operation.

This change eliminates 16 divider instances from the vector lanes, replacing them with a single reciprocal unit in the Scalar Core.

We evaluated three division methods: Radix-4 SRT [7], Newton-Raphson [8], and Goldschmidt [9]. We prioritized throughput against area cost. The prior non-restoring division was accurate but slow and not pipelined. Python simulations evaluating BF16 values identified the Goldschmidt method as the optimal balance of speed, accuracy, and area.

The initial guess accuracy heavily influences the Goldschmidt algorithm’s final result. We evaluated two derivation methods: a Constant Subtraction trick and a Lookup Table (LUT). The subtraction approach requires less area but yields a worse approximation (2 ULP maximum error). Because the reciprocal operation eliminates the numerator multiplier, sufficient area was freed to implement a 16-entry LUT indexed by the denominator’s mantissa MSBs. Python simulations confirmed the LUT achieves a 1 ULP maximum error (0.65 average) after two convergences, leading to the rejection of the subtraction approach.

Model Type	Activation/Norm Types	Hardware Operations Needed
CNN (e.g., ResNet) [3]	ReLU	Compare, Mux
Transformer [4]	ReLU / GELU, Softmax, LayerNorm	Exp, Reciprocal, Sq. Root, Reduction
Vision Transformer (ViT) [5]	GELU, Softmax, LayerNorm	Exp, Reciprocal, Sq. Root, Reduction
RNN (e.g., LSTM) [6]	Tanh, Sigmoid	Exp, Reciprocal

Table 1: Target AI Workloads and Required Vector Operations

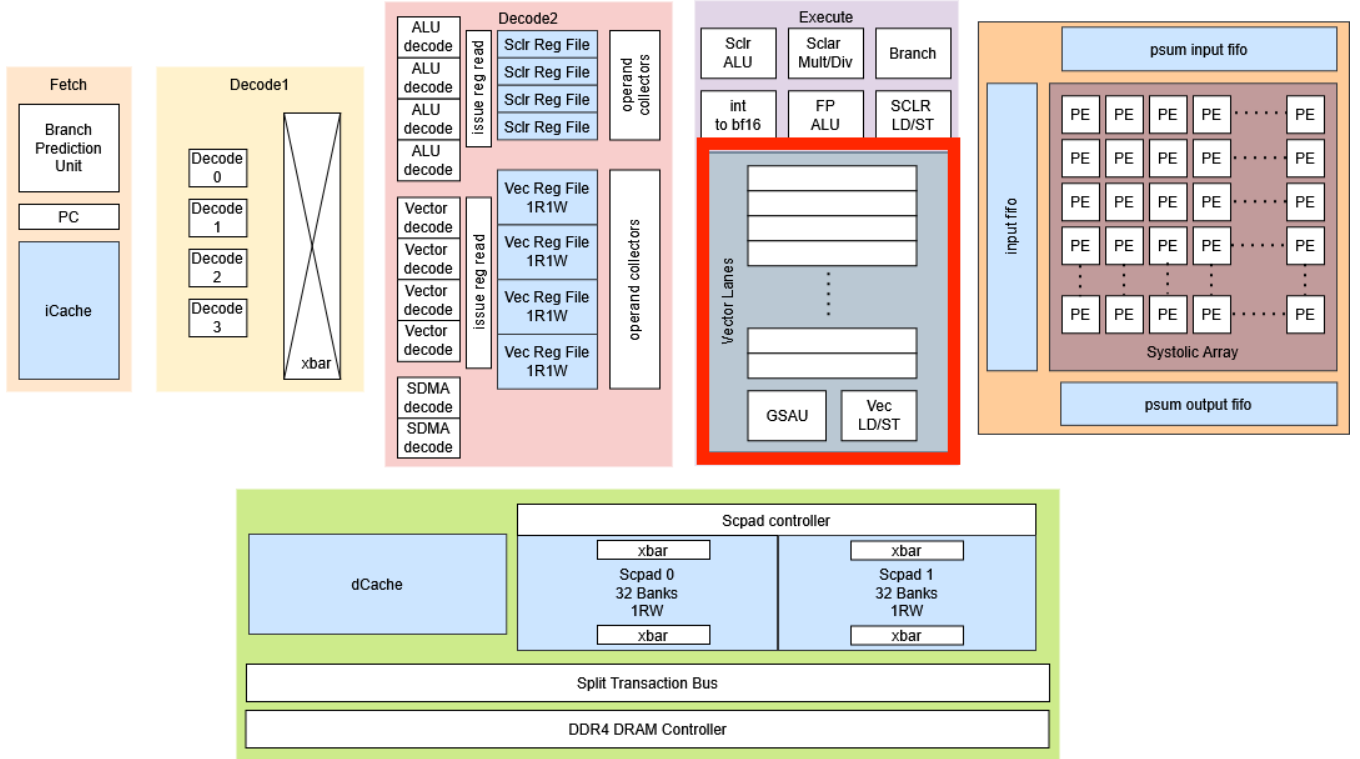


Figure 1: Atalla Ax01 Architecture

The final consideration was the iteration count. Each additional Goldschmidt iteration adds a multiplier-subtractor pair to the pipeline. Python simulations showed that with BF16’s 7-bit mantissa, accuracy plateaus after two iterations. Additional iterations consume area without improving ULP.

2.3 Exponential Unit

The exponential function, e^x , is a key component of AI operations such as Softmax, Sigmoid, and GELU. The prior Vector Core’s exponential unit used a multi-cycle Taylor-series approximation that issued at 14 cycles per result with 8 ULP worst-case error. We redesigned this per-lane unit with a LUT-based combinational datapath, achieving single-cycle issue and 2 ULP worst-case error while continuing to exploit the DLP in these activation functions.

To implement this unit, two potential designs were evaluated. First, a design based on Schraudolph’s method using piecewise polynomial approximations by Belano et al. [10]. Second, a LUT-based design by Jamro, Wiatr, and Wieglosz

[11]. While Schraudolph’s method avoids the need for a LUT, it instead requires additional integer multipliers. The LUT method allows for the use of a single multiplier while relying on the table for the remaining calculations. Ultimately, the LUT approach was selected due to the lower number of multipliers.

2.4 Transpose Unit

Matrix transpositions are necessary for many AI operations, such as aligning dimensions for matrix multiplication or calculating self-attention scores. Without a transpose unit, reading column-major data from row-major memory causes strided accesses that throttle memory bandwidth. A hardware transpose unit reorganizes the data so the vector core can perform contiguous reads.

The previous architecture tied the transpose operation directly to the scratchpad through in-scratchpad swizzling, which forced NUM_BANKS = MVL and prevented the scratchpad from being banked with wider per-bank ports.

To remove this constraint, we implemented an independent SRAM-based Transpose Unit.

A key constraint is that data arrives one vector per clock cycle, not as a complete matrix. The transposition algorithm must therefore apply the permutation incrementally as vectors arrive, avoiding the cost of buffering the entire matrix. We evaluated two candidate architectures from Lu [12]: a register-based approach and an SRAM-based approach. The register-based method uses a transposition algorithm that swaps and exchanges rows to achieve a transpose. While this avoids SRAM overhead, the swap pattern results in complex control logic and higher cycle counts, especially for non-square matrices. The SRAM-based approach was selected for its simpler control flow and the fact that both SRAM bank and CLOS network modules already existed in our codebase, reducing implementation risk and development time.

3 Micro-architecture

3.1 Top-Level

The integrated Vector Core, shown in fig. 2, centers on a banked Vector Register File (VRF, labeled VEGGIE in the figure) that supplies operands to all execution units. Decoded instructions enter from the Scheduler, vector operands are read from the VRF, and per-element mask bits are distributed to the lanes by the Mask Unit.

Element-wise operations dispatch to 16 parallel lanes (Lane0–Lane15), each operating on a 2-element slice of a 32-element vector. Per-functional-unit global result collectors gather slice outputs from all 16 lanes and reassemble the full vector before forwarding it to the Write Back Arbiter and WB Buffer, which writes the result back into the VRF. This replaces the per-lane writeback arbitration of the prior architecture.

The Vector Core exchanges data with two external paths. The Global Systolic Array Unit (GSAU) interfaces with the Systolic Array, transferring weights, partial sums, and activations. The four Vector Load/Store Units (VLSUs) interface with the four scratchpad channels, providing one independent load/store path per scratchpad. The Reduction Tree (RTree) consumes lane outputs for sum, min, and max reductions.

All inter-unit transfers use the valid/ready handshake protocol introduced in the Common Concepts section. This allows units with different latencies to interoperate without explicit timing schedules.

3.2 Reciprocal Unit

The Goldschmidt Division Algorithm [9] is an iterative method that repeatedly multiplies both the numerator and denominator by correction factors until the denominator converges to 1.0. Because the numerator of a reciprocal is always 1.0, we were able to eliminate the multiplier previously used for the numerator in the original divider design. To maximize throughput, the reciprocal unit was designed as a fully pipelined 9-stage architecture, shown in fig. 3. The datapath

instantiates two 1-cycle Wallace tree multipliers and one 2-cycle subtractor.

The execution flow is structured as follows:

- **Stages 1–2:** The initial exponent difference is calculated, and the input is multiplied by the LUT initial guess.
- **Stages 3–5:** The denominator traverses the internal registers and exits the subtractor to generate the next iteration’s guess factor.
- **Stages 6–7:** The result from stage 2 is multiplied by the new guess from stage 5.
- **Stages 8–9:** The final exponent is calculated and the result is latched for output.

For traffic control, the unit handles back-pressure from downstream consumers. When the downstream consumer is not ready, the pipeline enable signals are driven low, stalling the pipeline and propagating the stall to the upstream producer.

3.3 Exponential Unit

The exponential unit uses the LUT-based design [11], a method which avoids multiple expensive multipliers for calculating the mantissa output. This is effectively computed by

$$e^x \rightarrow 2^{\log_2(e) \cdot x}$$

The exponent, $\log_2(e) \cdot x$, can be split into its integer and fractional components: $I + F$. The integer part of the exponential, $2^I, I \in \mathbb{Z}$, can be easily computed in hardware using a barrel shifter, while the remaining fractional part of the exponent, $2^F, F \in [0, 1)$, is handled by the LUT. The values of the LUT are determined by $LUT_i = \text{mantissa bits of } 2^{\left(\frac{i}{N}\right)}$ where N is the size of the LUT. This unit is fully combinational and can process a new input every cycle. The calculation is done as follows:

1. The BF16 input’s mantissa needs to be converted to a fixed point representation, adjusting for subnormals as needed. Simultaneously, the input’s exponent bits will be unbiased to be used as the shift amount.
2. Next, the adjusted mantissa will be multiplied by a fixed point representation of $\log_2(e)$.
3. This product will then be shifted by the shift amount calculated earlier. The shift amount is clamped to avoid overflow errors.
4. Then the upper bits of the shifted product are extracted and rounded, with the sign bit from the input used to negate as needed. This fixed point value will then be decomposed into its integer and fractional components.
5. The integer component will be biased to provide the final exponent. The fractional component is used to index into the LUT, with the output of the LUT representing the final mantissa.

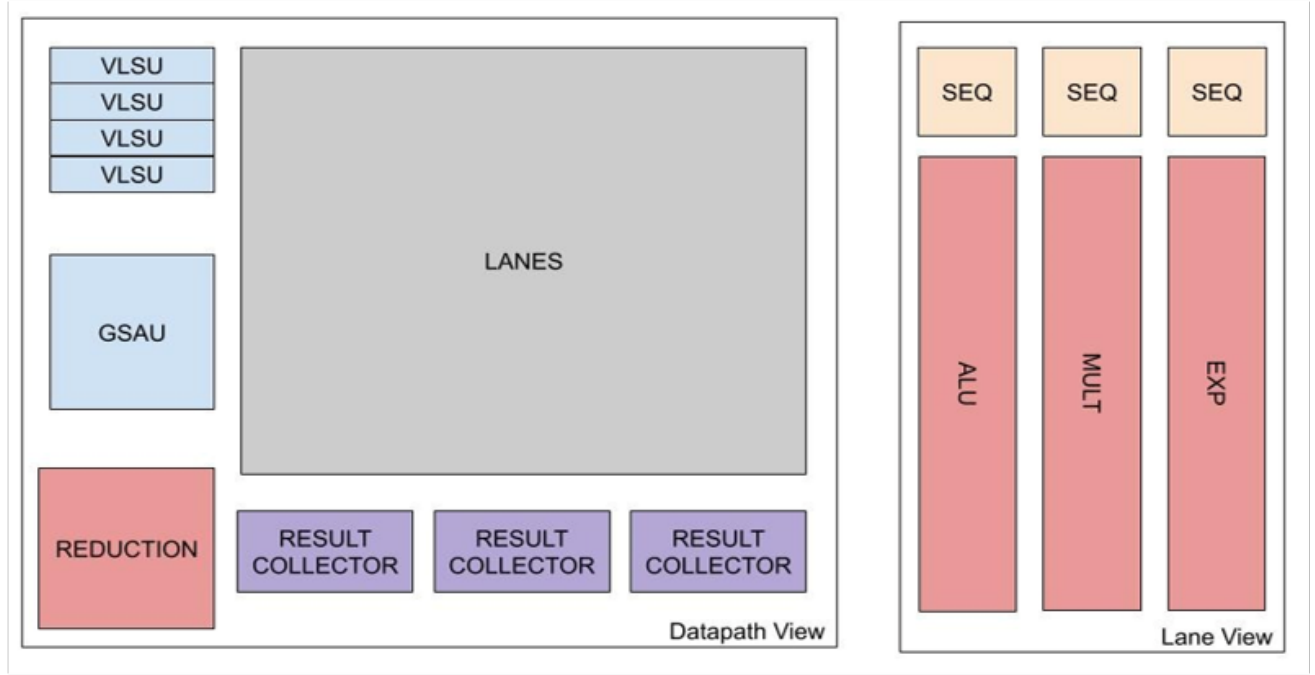


Figure 2: Vector Core Architecture

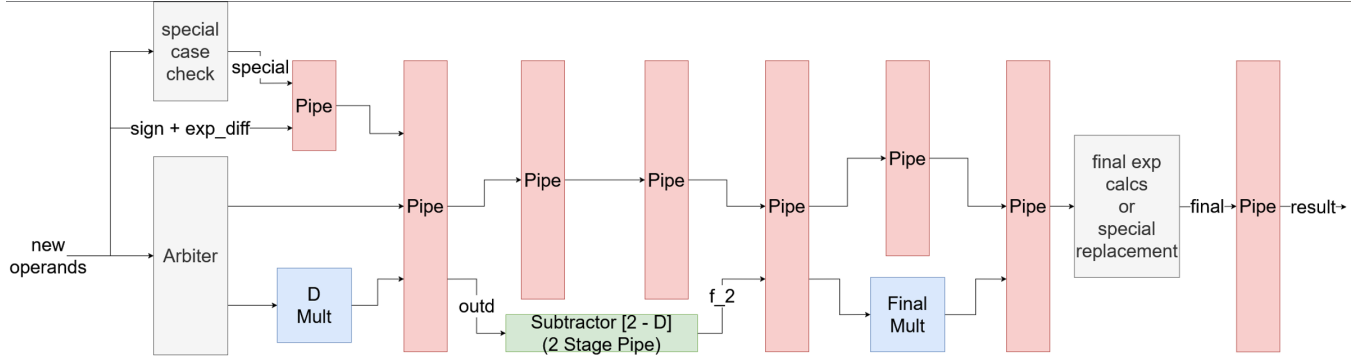


Figure 3: Reciprocal Unit Block Diagram

6. Finally, the result is assembled from its individual components. During this final step, we also account for special cases such as NaN and Inf.

The size of the LUT determines the ULP distribution. A 128 entry LUT was chosen because it allows for a worst case ULP error of 2. While increasing the LUT size to 256 allows for a worst case ULP error of 1, the extra area needed for the larger LUT was deemed to not be worth the area cost. Additionally, this unit implements the valid ready protocol which allows for robust handling of backpressure and starvation. The full datapath is shown in fig. 4.

3.4 Transpose Unit

The datapath, shown in fig. 5, utilizes a single shared 32×32 CLOS network and 32 independent SRAM banks. The CLOS

input is muxed by state. During writes, it routes incoming vectors from the interface and during reads, it routes data from the SRAM banks. The barrel-shift amount is computed from a vector counter to place each element into its transposed position.

The unit is controlled by a 7-state FSM that manages the multi-cycle latencies of the CLOS network (2 cycles) and SRAM:

- **IDLE:** Awaits a push or pop command via the interface.
- **WAIT_CLOS_WRITE** → **BUSY_WRITE:** On push, the input vector enters the CLOS network. After 2 cycles of CLOS routing, the shifted data is written into the SRAM banks. Once the write completes, the FSM returns to IDLE to handshake the next input vector. During this process, `ready_in` is asserted low.

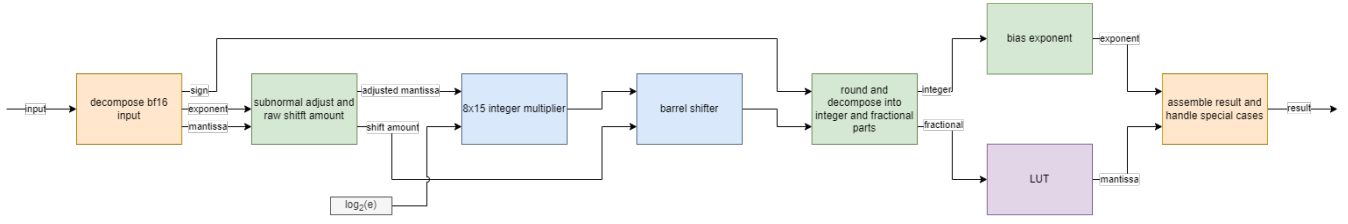


Figure 4: Exponential Unit Block Diagram

- **POPPING** → **WAIT_SRAM** → **WAIT_CLOS_READ** → **DONE**: On pop, the SRAM banks are read, the data passes through the CLOS network with a reverse shift, and the transposed vector is presented at the output. The DONE state holds `valid_out` high until the downstream consumer asserts `ready_out`, then automatically advances to the next vector until all 32 have been popped.

One structural trade-off of this architecture is that it requires exactly 32 vectors to be popped sequentially. If the target matrix has a vector length less than 32, the missing columns must be padded with zeros. However, modern AI workloads process massive matrices (e.g., 4096×4096) by breaking them down into 32×32 tiles. Therefore, this penalty is only incurred on the tiles at the edges of the matrix.

3.5 Vector Load Store Unit

The vector core features 4 Vector Load Store Units (VLSU), one for each scratchpad channel. The VLSU, shown in fig. 6, facilitates memory requests between the scratchpad and the vector register file (VRF). If a store instruction is received from the scheduler, the VLSU will fetch the data from the VRF and pass it to the scratchpad, along with the base scratchpad address, number of columns, and row ID. If the instruction is a load, the VLSU will initiate a memory request to the scratchpad and wait for the data. As a request is being sent, the register destination is saved in the load queue. The load queue is sized to match the length of the scratchpad latency plus 1, currently the latency is 12 at steady state. This allows a load request to be sent every cycle, as the initial request will arrive and be popped when the last FIFO slot is being filled, assuming no stalls. When the initial load request completes in the scratchpad, the data is sent to the VLSU, the register destination is popped from the load queue, and they are both sent to the write back buffer. If write back is not ready to accept the request, then the data is placed into a skid buffer, the load queue is not popped, and the scratchpad is told to stall until write back is no longer stalled.

4 Results

4.1 Top Level Results

The Vector Core was synthesized on the FreePDK 45-nm process node at 800 MHz, meeting timing closure on the integrated datapath consisting of the 16 vector lanes, 4 VLSUs,

the GSAU, the Reduction Tree, the Transpose Unit, and the Vector Register File. As shown in table 2, the integrated design consumes 0.1767 mm^2 of cell area at 2277.48 mW total power. table 3 reports the cell-size distribution, with 91.43% of the 90,764 standard cells at $1 \times$ drive strength.

Unit	Cell Area (mm^2)	Power (mW)	Count	Percent of Total
ALU	.0028	2.54	-	-
Multiplier	.0018	3.29	-	-
Exp	.0020	2.51	-	-
Lane	.0066	9.48	16	59.8%
Reduction	.0204	22.53	1	11.5%
VLSU	.0059	5.82	4	13.4%
GSAU	.0127	7.26	1	7.2%
Transpose	.0379	33.52	1	21.4%
Top Level	.1767	2277.48	1	100%

Table 2: Area and Power Breakdown (FreePDK 45-nm, 800 MHz)

Cell Size	Count	Percent
1x	82988	91.43
2x	5792	6.38
4x	1872	2.06
8x+	112	0.12

Table 3: Top Level Cell Size Breakdown

4.2 Reciprocal Unit Results

A SystemVerilog testbench verified the reciprocal unit across all possible denominator inputs (65,536 BF16 inputs) against a PyTorch golden model, tracking Unit in the Last Place (ULP) errors. It successfully evaluated edge cases ($\pm\infty$, ± 0 , NaNs), power-on resets, mid-flight flushes, and random input starvation without data corruption.

As shown in table 4, the unit achieves a maximum ULP error of 1 and an average ULP error of 0.65. Out of all tested values, over 67% returned a perfect result with 0 ULP error.

The fully pipelined design achieves a throughput of 1 instruction per cycle (Effective CPI of 1.0). When synthesized for the FreePDK 45-nm process node at 800 MHz, the reciprocal unit consumes $5,064 \mu\text{m}^2$ of total area ($3,139 \mu\text{m}^2$ cell, $1,925 \mu\text{m}^2$ net) at 4.54 mW. Because the reciprocal unit resides in the scalar core as a single instance rather than being replicated per vector lane, the total system area is reduced.

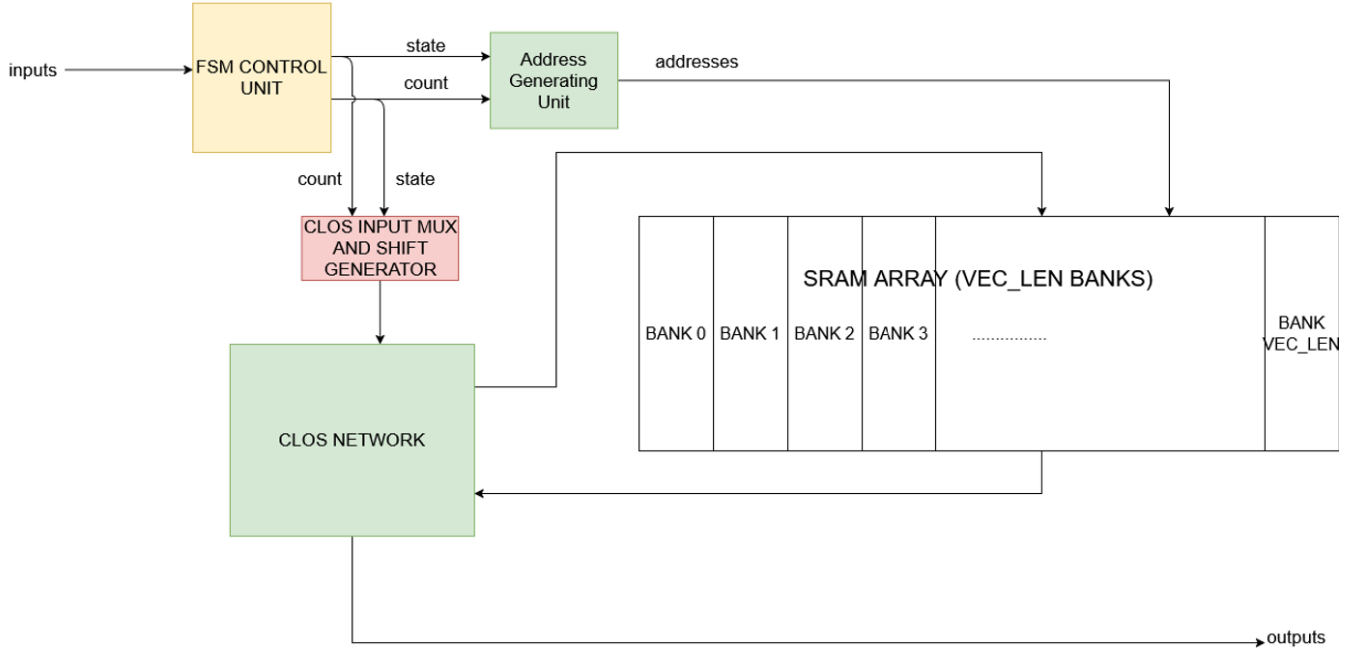


Figure 5: Transpose Unit Block Diagram

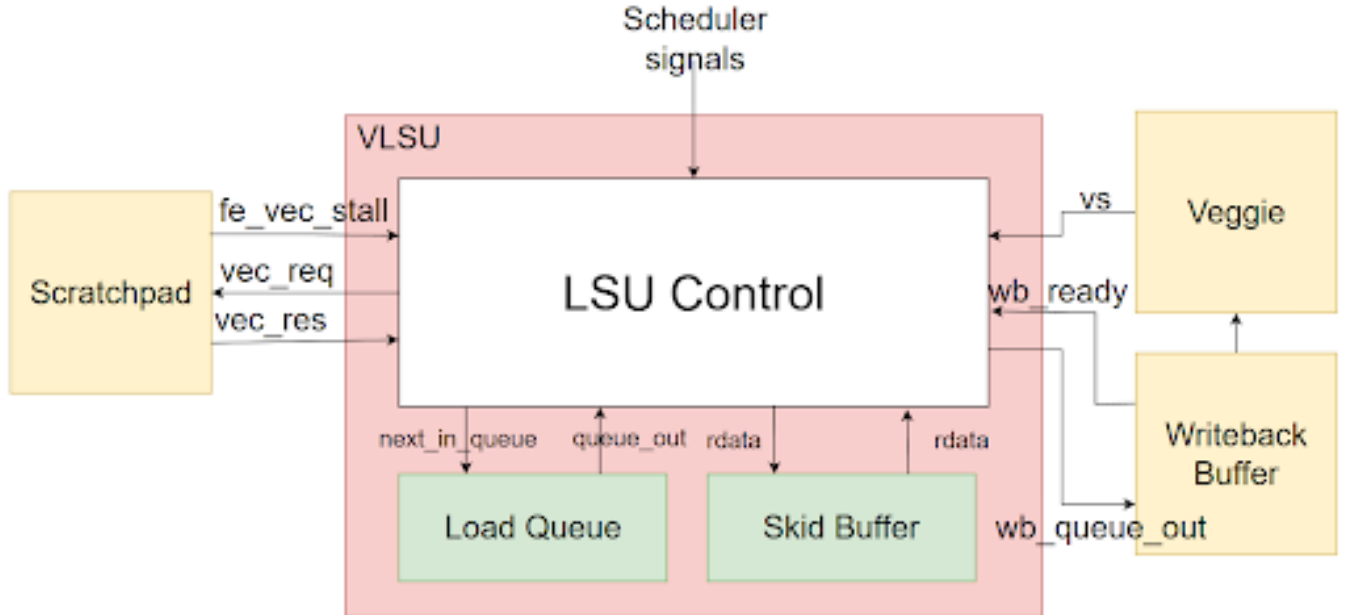


Figure 6: Vector Load Store Unit Block Diagram

4.3 Exponential Unit Results

Similar to the reciprocal unit, a testbench swept over all 65,536 possible BF16 inputs, comparing outputs to a golden model calculated using PyTorch. As shown in table 5, the unit achieves a worst case ULP error of 2 with an average of 0.03. Nearly 97% of cases result in 0 ULP error.

Since the calculations are done fully combinationaly, the exponential unit is able to process a new input every cycle. After synthesizing on FreePDK 45nm targeting 800MHz, we find that the unit takes $1798 \mu m^2$ total area.

4.4 Transpose Unit Results

Pushing a matrix requires $N \times (2 + S_1)$ cycles, where N is the number of input vectors and S_1 is the SRAM write latency. Popping the transposed matrix requires $32 \times (2 + S_2)$ cycles, where S_2 is the SRAM read latency.

Synthesized on FreePDK 45-nm at 800 MHz, the unit consumes $37,923 \mu m^2$ of logic area. PRACTI sweeps estimate the 32 SRAM banks consume an additional $59,065 \mu m^2$. Power consumption is dominated by dynamic switching (33.11 mW of the 33.52 mW total). While this dedicated unit introduces

Metric	Value
0 ULP Cases	44,368
1 ULP Cases	21,168
2 ULP Cases	0
Average ULP	0.65
Max ULP	1
Total Area (μm^2)	5063.571
Effective CPI	1.0

Table 4: Reciprocal Unit Performance and Accuracy Results (FreePDK 45-nm, 800 MHz)

Metric	Value
0 ULP Cases	63,549
1 ULP Cases	1,977
2 ULP Cases	10
Average ULP	0.03
Max ULP	2
Total Area (μm^2)	1798.363
Effective CPI	1.0

Table 5: Exponential Unit Performance and Accuracy Results (FreePDK 45nm, 800 MHz)

local SRAM overhead, decoupling transposition from the primary memory subsystem eliminates the scratchpad banking constraint, enabling wider read ports and higher overall system bandwidth.

A testbench evaluated all 32 matrix row configurations (from 1×32 up to 32×32), validating handling of irregular sizes and adherence to the Valid/Ready protocol. The unit passed all tests without data corruption or protocol violations.

4.5 Vector Load Store Unit Results

A SystemVerilog testbench exercised two VLSU instances against two real scratchpads, running 34 directed tests covering pipeline stress, load-queue saturation, writeback backpressure, skid-buffer behaviors, dual-channel parallel operation, store-load integrity, multi-row addressing, RAW hazards, and sustained mixed traffic patterns. All tests passed without data corruption, deadlock, or valid-ready protocol violations.

The VLSU’s load queue is sized to match the measured scratchpad latency plus one (FIFO_DEPTH = 13, SP_LATENCY = 12), allowing one load request to issue per cycle in steady state. Test T34 confirmed sustained one-load-per-cycle throughput on back-to-back loads.

As shown in table 2, a single VLSU instance occupies 0.0059 mm^2 of cell area at 5.82 mW when synthesized for the FreePDK 45-nm process node at 800 MHz. Including net area, total area per VLSU is $10,238 \mu m^2$ ($5,959 \mu m^2$ cell, $4,280 \mu m^2$ net). The cell area is dominated by sequential elements ($4,174 \mu m^2$, 70% of cell area), primarily the load queue, skid buffer, and request-state registers. Across the four VLSU instances in the integrated Vector Core, total VLSU area is approximately $40,952 \mu m^2$ at 23.28 mW combined.

5 Discussion of Results

5.1 Top Level

The top level integration of the vector core has met expectations. As shown in table 2, the design is well under the maximum area target of 0.25 mm^2 . The design meets the 800 MHz clock target without using an unnecessary amount of large cells as shown by table 3. Notably, the sum of the individual unit area percentages exceeds 100% (approximately 113.3%). This is an expected artifact of synthesis, where the top-level area shrinks relative to the sum of its isolated parts due to cross component logic optimization.

5.2 Reciprocal Unit

The reciprocal unit’s accuracy met expectations. Python simulations predicted a maximum ULP of 1 with a 16-entry LUT, and the hardware results confirmed this exactly. The 67% rate of perfect (0 ULP) results was notable given BF16’s limited 7-bit mantissa [1]. The ULP regression from 0 (prior non-restoring divider) to 1 is an acceptable trade-off for AI workloads, where the BF16 format already prioritizes range over precision.

At $5,064 \mu m^2$, the unit is substantially smaller than the 90-nm Goldschmidt divider alternatives explored earlier ($18,433$ to $22,283 \mu m^2$). Architecturally, eliminating the numerator multiplier and consolidating to a single scalar-core instance contribute to this reduction beyond what the 45-nm process shrink alone would provide.

5.3 Exponential Unit

The exponential unit’s worst case ULP error was as expected based on the software simulation. The large number of cases where ULP error is 0 is helped by that fact that e^x quickly gets squashed to infinity for larger values of x due to the limited range of the BF16 format. Compared to the prior multi-cycle iterative design’s 8 ULP worst-case error, the LUT-based re-design delivers an accuracy improvement to 2 ULP.

Interestingly, the synthesized area of $1798 \mu m^2$ is quite small compared to the prior design’s $\sim 16,000 \mu m^2$ on 90-nm. Partially, this is because the 15 bit signal going into the 8×15 multiplier is a constant, allowing for aggressive optimization. This leaves the barrel shifter and the LUT as the largest components within the unit. Ultimately, the exponential unit’s performance is well within expectations.

5.4 Transpose Unit

The transpose unit’s area is dominated by SRAM: $59,065 \mu m^2$ of SRAM versus $37,923 \mu m^2$ of logic. This was expected for a design with 32 independent SRAM banks. The power profile confirms this, as dynamic power (33.11 mW) accounts for over 98% of total power (33.52 mW), consistent with expectations for an SRAM-heavy design.

5.5 Vector Load Store Unit

Registers and flip-flops (the load queue, skid buffer, and request-state tracking) account for 70% of cell area and over 80% of internal power, while combinational logic, primarily address generation and handshake control, contributes 25% of the area.

Sizing the load queue to match scratchpad latency plus one was a deliberate choice based on testbench performance analysis. This confirmed that the depth is exactly what is required to sustain one load per cycle without backpressure. A shallower queue would force the scratchpad to stall once the queue filled in flight, a deeper queue would consume more area without improving throughput.

6 Limitations and Future Work

6.1 Top Level

End-to-end performance benchmarking on full activation kernels is incomplete. While individual functional units have been verified for correctness and synthesized for area and power, cycle-counts on representative workloads (e.g., Softmax, GELU, full LayerNorm) have not been measured at the integrated datapath level. The testbench performance monitor described in this report provides the infrastructure to gather this data; running it across a workload suite is future work.

6.2 Reciprocal Unit

The move from a full divider to a reciprocal unit means true division now requires two operations: a scalar reciprocal followed by a vector multiply. For workloads dominated by pure division rather than normalization, the reciprocal result must traverse writeback, be broadcast and stored in the VRF, and then be consumed by a separate vector multiply instruction, which adds several cycles to the critical path compared to a hypothetical single-instruction divide. Additionally, the single scalar instance could become a throughput bottleneck under sustained back-to-back reciprocal requests, though the fully pipelined design mitigates this for most practical workloads. Future work could explore adding a skid buffer or implementing a second reciprocal lane. Alternatively, a full divider could replace the reciprocal unit at the cost of additional area.

6.3 Exponential Unit

Two future directions remain for the exponential unit. First, the LUT could be shrunk below 128 entries to reduce area at the cost of higher worst-case ULP error. A tradeoff study would identify the optimal size for AI workloads. Second, the unit is fully combinational and sets the lane's critical path; pipelining it would enable higher clock frequencies at the cost of additional latency.

6.4 Transpose Unit

The fixed 32×32 tile size means the unit cannot be partially utilized. For example, even a 4×32 matrix occupies the full SRAM capacity and requires 32 pop cycles. For workloads with irregular matrix dimensions, this overhead becomes more apparent. Additionally, a transposed matrix must be fully popped before the next matrix can be pushed, creating a pipeline bubble between consecutive transpositions. Future iterations could explore utilizing these SRAM banks more efficiently.

6.5 Vector Load Store Unit

The load queue depth is fixed at 13 entries, tuned to the current 12-cycle scratchpad latency. If scratchpad latency increases in the future, the queue must be resized to maintain one-load-per-cycle throughput. The skid buffer is also single-entry, which is sufficient for the current latency but may need a larger structure if downstream backpressure becomes more frequent.

6.6 MxN

The current Vector Core contains 16 lanes, each operating on 2 elements to fully process a 32-element vector. This is extracting DLP through the use of SIMD instructions. However, since AI workloads are highly regular and many of the same instructions are issued in batches across a group of vectors, there exists an opportunity to extract more DLP. We propose a new approach to the Vector Core, where the lanes have separate sublanes, each containing a smaller local register file and functional units. A benefit of this approach is that each sublane can stream a vector into the systolic array every cycle. This approach mirrors the lane/sublane architecture of Google's TPU [13, 14]. A clear drawback of this approach is that there would be under-utilization of the vector core when the workload isn't fulfilling all the M sub-lanes. With this new approach, there's also a benefit of reducing the monolithic crossbar between vector lanes and the register file in our current vector core design, replacing it with a multitude of smaller crossbars. To properly determine if this approach would benefit our core in terms of PPA, more studies on power sweeps and area sweeps must be done. Although the crossbars may be smaller, there may be an alternative cost of switching or leakage power.

6.7 Pooling

Pooling is commonly utilized in many convolutional neural networks (CNNs), including ResNet [3] and AlexNet [15], along with certain vision transformers. Our current Vector Core architecture does not support these pooling operations efficiently, as it lacks any dedicated hardware for the operation. To address this, we propose to modify existing hardware of the per-lane ALUs to support for pooling functionality, however this approach still requires further development as it does not account for the varying stride the pooling may use. Further design analysis and research into how pooling is done on

other existing hardware is needed to properly identify the best solution to support this operation.

6.8 Four VLSUs May Exceed Workload Demand

The four VLSUs match the four scratchpad channels but may exceed typical workload demand. Most AI activation kernels execute as vector-wide load $\rightarrow$ compute $\rightarrow$ store sequences without sustained four-channel parallelism. With four VLSUs, simultaneous loads can claim all four of the VRF's data write ports, forcing the VRF to serialize against lane and GSAU writebacks. The arbitration crossbar between VLSU outputs and VRF write ports grows with the VLSU count. A workload study comparing 2-VLSU and 4-VLSU configurations would identify whether the additional memory parallelism justifies the VRF arbitration complexity and area.

7 Conclusion

In this work, improvements to the Atalla Vector Core addressed compute and memory bottlenecks for AI workloads. By moving from isolated per-lane arithmetic units to centralized and pipelined execution logic, the Vector Core reduces area while sustaining higher throughput on data-parallel activation functions. At the system level, the Vector Load Store Unit (VLSU) and dedicated Transpose Unit are both integrated into the top-level datapath. The Transpose Unit decouples data reorganization from the primary Scratchpad, removing the constraint that previously prevented the scratchpad from being banked with wider per-bank read ports. Top-level synthesis on FreePDK 45-nm at 800 MHz fits the integrated Vector Core in 0.1767 mm^2 , 71% of the 0.25 mm^2 area budget. Future development will focus on profiling end-to-end AI workloads to identify remaining architectural bottlenecks and underutilized datapaths. This will guide future improvements like the proposed $M \times N$ sub-lane architecture to increase data-level parallelism. It will also motivate adding memory-striding support for pooling operations.

References

- [1] S. Wang and P. Kanwar, "BFLOAT16: The secret to high performance on cloud TPUs," Google Cloud Blog, 2019. [Online]. Available: <https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus>
- [2] ARM, *AMBA AXI and ACE Protocol Specification*, 2023, aRM IHI 0022H. [Online]. Available: <https://developer.arm.com/documentation/ihi0022/latest>
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] A. Dosovitskiy, L. Beyer, A. Kolesnikov *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.
- [6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] S. F. Oberman and M. J. Flynn, "An analysis of division algorithms and implementations," Stanford University, Tech. Rep., 1995. [Online]. Available: <http://i.stanford.edu/pub/ctr/reports/csl/tr/95/675/CSL-TR-95-675.pdf>
- [8] M. D. Ercegovac and T. Lang, *Digital Arithmetic (The Morgan Kaufmann Series in Computer Architecture and Design)*. Morgan Kaufmann, 2004.
- [9] T. D. Roy, "Implementation of Goldschmidt's algorithm with hardware reduction," *arXiv preprint arXiv:1909.10154*, 2019.
- [10] A. Belano, Y. Tortorella, A. Garofalo, L. Benini, D. Rossi, and F. Conti, "A flexible template for edge generative ai with high-accuracy accelerated softmax & gelu," *arXiv preprint arXiv:2412.06321*, 2024.
- [11] E. Jamro, K. Wiatr, and M. Wielgosz, "Fpga implementation of 64-bit exponential function for hpc," in *Proceedings of the 2007 IEEE*. IEEE, 2007, pp. 718–721.
- [12] Y. Lu, "Matrix transposition in hardware: Challenges and approaches," Master's thesis, Lund University, 2024. [Online]. Available: <https://www.diva-portal.org/smash/get/diva2:2005914/FULLTEXT01.pdf>
- [13] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1–12. [Online]. Available: <https://arxiv.org/abs/1704.04760>
- [14] T. Norrie, N. Patil, D. H. Yoon, G. Kurian, S. Li, J. Laudon, C. Young, N. Jouppi, and D. Patterson, "The design process for Google's training chips: TPUv2 and TPUv3," *IEEE Micro*, 2021.
- [15] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, 2012.

A Teamwork

A.1 [Julio Hernandez]

This semester, I completed the SystemVerilog implementation of the Vector Load Store Unit (VLSU), whose architecture was established in the prior semester. Verification work included a 34-test SystemVerilog test bench that used two VLSU instances against two real scratchpads, covering pipeline stress, load-queue saturation, writeback backpressure, skid-buffer behaviors, dual-channel parallel operation, and sustained mixed traffic patterns.

For the report, I authored the Vector Load Store Unit subsections in Micro-architecture, Results, Discussion, and Limitations. The Top-Level subsection of Micro-architecture, and the Top Level and Four VLSUs subsections of Limitations. I rewrote the Abstract and did general editing for technical accuracy and clarity.

A.2 [Jacob Walter]

This semester, I helped with the top-level RTL integration of the Vector Core, updated the per-lane functional units, and helped with the base C++ infrastructure for the SystemVerilog/DPI testbench used in integration verification. I also ran the synthesis flow on FreePDK 45-nm at 800 MHz to produce the area, power, and cell-size numbers reported throughout the Results section.

For the report, I populated the area, power, and cell-size tables in the Top Level Results subsection and helped author the Top Level subsection in Discussion of Results.

A.3 [Vedant Sharma]

This semester, I helped with the top-level integration of Vector Core, worked on certain functional units and datapath units, along with the full integration and verification between Vector Core, Systolic Array, and Scratchpad using DPI-C-driven testbenching. I also created test cases and kernel for simulating how an actual workload would perform on our Vector Core.

For this report, I wrote the MxN limitations and future work section, the Pooling limitations and future work, edited the Introduction, and did general work in editing and restructuring the report.

A.4 [Mayank Patibandla]

This semester, I implemented the per-lane exponential unit's redesigned LUT-based combinational datapath, which achieves single-cycle issue and a 2 ULP worst-case error against the prior multi-cycle design's 8 ULP. I also developed the performance counters for the lane and GSAU wrappers, which form the basis of the testbench performance monitor used to gather per-unit utilization and stall data.

For the report, I authored the Exponential Unit subsections in Design Choices, Micro-architecture, Results, Discussion, and Limitations.

A.5 [Akhil Yada]

This semester, I worked jointly with Brian Zhuang on the design and verification of the reciprocal unit and the dedicated Transpose Unit. This work spanned algorithm exploration, RTL implementation, and standalone verification testbenches for both units.

For the report, I authored the initial drafts of the Introduction subsections. Brian and I co-authored the Reciprocal Unit and Transpose Unit subsections in Design Choices, Micro-architecture, Results, Discussion, and Limitations.

A.6 [Brian Zhuang]

This semester, I worked jointly with Akhil Yada on the design and verification of the reciprocal unit and the dedicated Transpose Unit. This work spanned algorithm exploration, RTL implementation, and standalone verification testbenches for both units.

For the report, Akhil and I co-authored the Reciprocal Unit and Transpose Unit subsections in Design Choices, Micro-architecture, Results, Discussion, and Limitations.